VISVESVARAYATECHNOLOGICALUNIVERSITY

"JnanaSangama",Machhe,Belagavi,Karnataka-590018

Lab Experiment Record

Project Management with Git [BCSL358C]

SubmittedinpartialfulfillmenttowardsAECof3rdsemesterof

Bachelorof Engineering
in
ComputerScienceandEngineering
(ArtificialIntelligence&MachineLearning)

Submittedby

SOWNDARYA B
4GW24CI049

DEPARTMENTOF CSE(ArtificialIntelligence&MachineLearning)

GSSSINSTITUTEOFENGINEERING&TECHNOLOGYFORWOMEN

(AffiliatedtoVTU,Belagavi,ApprovedbyAICTE,NewDelhi&Govt.ofKarnataka)

K.R.SROAD,METAGALLI,MYSURU-570016,KARNATAKA

INDEX

SR.NO.	CONTENTS	Page No.
1.	Experiment 1	10
2.	Experiment 2	12
3.	Experiment 3	14
4.	Experiment 4	16
5.	Experiment 5	18
6.	Experiment 6	20
7.	Experiment 7	21
8.	Experiment 8	22
9.	Experiment 9	23
10.	Experiment 10	24
11.	Experiment 11	26
12.	Experiment 12	27

SYLLABUS

1. **Setting Up and Basic Commands**

Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.

2. **Creating and Managing Branches**

Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."

3. **Creating and Managing Branches**

Write the commands to stash your changes, switch branches, and then apply the stashed changes.

4. **Collaboration and Remote Repositories**

Clone a remote Git repository to your local machine.

5. **Collaboration and Remote Repositories**

Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

6. **Collaboration and Remote Repositories**

Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.

7. **Git Tags and Releases**

Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.

8. **Advanced Git Operations**

Write the command to cherry-pick a range of commits from "source-branch" to the current.

9. **Analyzing and Changing Git History**

Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?

10. **Analyzing and Changing Git History**

Write the command to list all commits made by the author “JohnDoe’’ between

"2023-01-01" and "2023-12-31."

11.Analyzing and Changing Git History

Write the command to display the last five commits in the repository's history.

12.Analyzing and Changing Git History

Write the command to undo the changes introduced by the commit with the ID "abc123".

Git Commands List

Git is a popular version control system used for tracking changes in software development projects. Here's a list of common Git commands along with brief explanations:

1. **git init**: Initializes a new Git repository in the current directory.
2. **git clone <repository URL>**: Creates a copy of a remote repository on your local machine.
3. **git add <file>**: Stages a file to be committed, marking it for tracking in the next commit.
4. **git commit -m "message"**: Records the changes you've staged with a descriptive commit message.
5. **git status**: Shows the status of your working directory and the files that have been modified or staged.
6. **git log**: Displays a log of all previous commits, including commit hashes, authors, dates, and commit messages.
7. **git diff**: Shows the differences between the working directory and the last committed version.
8. **git branch**: Lists all branches in the repository and highlights the currently checkedout branch.

9. **git branch <branchname>**: Creates a new branch with the specified name.
10. **git checkout <branchname>**: Switches to a different branch.
11. **git merge <branchname>**: Merges changes from the specified branch into the currently checked-out branch.
12. **git pull**: Fetches changes from a remote repository and merges them into the current branch.
13. **git push**: Pushes your local commits to a remote repository.
14. **git remote**: Lists the remote repositories that your local repository is connected to.
15. **git fetch**: Retrieves changes from a remote repository without merging them.
16. **git reset <file>**: Unstages a file that was previously staged for commit.
17. **git reset --hard <commit>**: Resets the branch to a specific commit, discarding all changes after that commit.
18. **git stash**: Temporarily saves your changes to a "stash" so you can switch branches without committing or losing your work.
19. **git tag**: Lists and manages tags (usually used for marking specific points in history, like releases).
20. **git blame <file>**: Shows who made each change to a file and when.
21. **git rm <file>**: Removes a file from both your working directory and the Git repository.
22. **git mv <oldfile> <newfile>**: Renames a file and stages the change.

These are some of the most common Git commands, but Git offers a wide range of features and options for more advanced usage. You can use git --help followed by the command name to get more information about any specific command, e.g., git

help commit.

EXPERIMENT–1: Git Repository Initialization and First Commit

Git is a distributed version control system used to track and manage changes in files. This experiment demonstrates the basic Git workflow such as repository creation, staging, and committing files.

Before starting version control operations, Git requires user identity configuration. In this step, the global user name and email are set, which will be linked to all commits created by the user.

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~ (master)
$ git init
Reinitialized existing Git repository in C:/Users/Sindhu B/.git/

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~ (master)
$ git config --global user.name "Sowndarya B"

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~ (master)
$ git config --global user.email "soundaryab0402@gmail.com"
```

Procedure with code and step description :

Step 1 : Create a New Directory

```
mkdir git_exp1
cd git_exp1
```

Step 2 : Initialize Git Repository

```
git init
```

This command initializes an empty Git repository by creating a

hidden –git folder to store version history.

Step 3 : Create a File

notepad file1.txt

A new text file is created to add content and demonstrate file tracking using Git.

Git Lab Experiment 1
Learning basic Git commands.

Step 4 : Check Repository Status

git status

This command displays the current state of the repository and shows that the file is untracked.

Step 5 : Add File to Staging Area

git add file1.txt

Step 6 : Commit the File

git commit -m "Added file1.txt for first Git experiment"

This command permanently saves the staged changes into the local repository with a commit message

Step 7 : View Commit History

git log

Displays the commit history including commit ID, author, date, and commit message.

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~ (master)
$ mkdir git_lab
mkdir: cannot create directory ‘git_lab’: File exists

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~ (master)
$ cd git_lab

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git init
Reinitialized existing Git repository in C:/Users/Sindhu B/git_lab/.git/

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ notepad file1.txt

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file1.txt

nothing added to commit but untracked files present (use "git add" to track)

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git add file1.txt
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git commit -m "Added file1.txt for first Git experiment"
[main 280441d] Added file1.txt for first Git experiment
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt
```

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git add .

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git log
commit 280441d04f26cb75087b5de44c2ee94272b0dd06 (HEAD -> main)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Tue Jan 6 15:13:42 2026 +0530

    Added file1.txt for first Git experiment

commit 07e4fcc30b28baa5d33638b359872fd2655f1305
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 21:42:12 2026 +0530

    Initial commit

commit 46f12f7831f383ad7c203c9f2272092af9ef6366 (origin/main, origin/HEAD)
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:25:46 2026 +0530

    Delete lab1.txt

commit 0b432ad866fc6ff03f99534e9d64e6e5413ee067
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:21:56 2026 +0530

    Initial commit -adding a new file

commit 389576d1cb86d11e43e719ee282b033d3df18d5b
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:19:40 2026 +0530

    Initial commit

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$
```

EXPERIMENT – 2: Creating And Managing Branches

Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."

Git branching allows developers to work on different versions of a project simultaneously without affecting the main code. This experiment demonstrates how to create and manage branches in a Git repository.

Procedure with code and step description :

Step 1 : Open Existing Git Repository

```
cd git_exp1
```

The existing Git repository from the previous experiment is opened to perform branch operations.

Step 2 : Create a New Branch

```
git branch feature1
```

A new branch named `feature1` is created to develop new features independently.

Step 3 : List Available Branches

```
git branch
```

This command displays all available branches and highlights the current active branch.

Step 4 : Switch to the New Branch

```
git checkout feature1
```

The working directory is switched from the main branch to the `feature1` branch.

Step 5 : Modify a File in New Branch

```
notepad file1.txt
```

The file is modified to show how changes can be made independently in a separate branch.

Step 6 : Add and Commit Changes

```
git add file1.txt
git commit -m "Updated file1.txt in feature1 branch"
```

The changes made in the new branch are staged and committed.

Step 7 : Switch Back to Main Branch

```
git checkout master
```

The repository is switched back to the main branch to merge changes.

Step 8 : Merge the Branch

```
git merge feature1
```

The changes from `feature1` branch are merged into the main branch.

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (main)
$ git branch feature1

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (main)
$ git branch
  feature1
* main

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (main)
$ git checkout feature1
Switched to branch 'feature1'

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ notepad file1.txt

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ git add file1.txt

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ git commit -m "Updated file1.txt in feature1 branch"
On branch feature1
nothing to commit, working tree clean

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ git checkout master
error: pathspec 'master' did not match any file(s) known to git

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ git branch
* feature1
  main

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature1)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (main)
$ git checkout master
error: pathspec 'master' did not match any file(s) known to git
```

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (main)
$ git checkout -b master
Switched to a new branch 'master'

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git merge feature1
Already up to date.

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git branch feature1
fatal: a branch named 'feature1' already exists

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git branch
  feature1
  main
* master

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git checkout feature1
Switched to branch 'feature1'

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature1)
$ notepad file1.txt

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature1)
$ git add file1.txt

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature1)
$ git commit -m "Updated file1.txt in feature1 branch"
On branch feature1
nothing to commit, working tree clean

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature1)
$ git checkout master
Switched to branch 'master'

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git merge feature1
Already up to date.

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$
```

EXPERIMENT – 3: Creating and Managing Branches

Write the commands to stash your changes, switch branches, and then apply the stashed changes.

A remote repository allows code to be stored and shared on a central server like GitHub. This experiment demonstrates how to connect a local Git repository to a remote repository and synchronize changes using push and pull commands.

Procedure with code and step description :

Step 1 : Check current status

```
git status
```

Displays the list of modified files that are not yet committed.

Step 2 : Stash the changes

```
git stash
```

Temporarily saves the uncommitted changes and cleans the working directory.

Step 3 : Switch to another branch

```
git checkout feature_branch
```

Switches from the current branch to another branch without losing changes

Step 4 : Apply the stashed changes

```
git stash apply
```

Restores the previously stashed changes in the current branch.

Step 5 : View stash list

```
git stash list
```

Displays all saved stashes.

Step 6 : Remove stash after applying

```
git stash drop
```

Deletes the applied stash from the stash list.

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git status
On branch master
nothing to commit, working tree clean

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git stash
No local changes to save

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git checkout feature_branch
error: pathspec 'feature_branch' did not match any file(s) known to git

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master)
$ git checkout -b feature_branch
Switched to a new branch 'feature_branch'

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git stash apply
No stash entries found.

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git stash list

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git add .

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$
```

EXPERIMENT – 4: Collaboration And Remote Repositories

Clone a remote Git repository to your local machine.

Cloning is the process of creating a local copy of an existing remote repository. This experiment demonstrates how Git clone is used to download a complete project along with its version history**.**

Procedure with code and step description :

Step 1 : Open Git Bash

Git Bash is opened to execute Git commands.

Step 2 : Select a Directory for Cloning

```
cd Desktop
```

The directory where the remote repository will be cloned is selected.

Step 3 : Copy Remote Repository URL

https://github.com/username/git_exp3.git

Step 4 : Clone the Remote Repository

```
git clone https://github.com/username/git_exp3.git
```

This command creates a local copy of the remote repository along with all files and commit history.

Step 5 : Check Repository Status

git status

Displays the status of the cloned repository and confirms successful cloning.

Step 6 : View Commit History

git log

Shows the complete commit history that was cloned from the remote repository.

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature_branch)
$ git clone https://github.com/soundaryab0402-cloud/git_lab.git
Cloning into 'git_lab'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (feature_branch)
$ git status
On branch feature_branch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git_lab/

nothing added to commit but untracked files present (use "git add" to track)
```

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git log
commit 280441d04f26cb75087b5de44c2ee94272b0dd06 (HEAD -> feature_branch, master, main, feature1)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Tue Jan 6 15:13:42 2026 +0530

    Added file1.txt for first Git experiment

commit 07e4fcc30b28baa5d33638b359872fd2655f1305
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 21:42:12 2026 +0530

    Initial commit

commit 46f12f7831f383ad7c203c9f2272092af9ef6366 (origin/main, origin/HEAD)
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:25:46 2026 +0530

    Delete lab1.txt

commit 0b432ad866fc6ff03f99534e9d64e6e5413ee067
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:21:56 2026 +0530

    Initial commit -adding a new file

commit 389576d1cb86d11e43e719ee282b033d3df18d5b
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:19:40 2026 +0530

    Initial commit

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ |
```

EXPERIMENT – 5: Collaboration and Remote Repositories

Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.

Git fetch is used to download the latest changes from a remote repository without merging them. Git rebase is used to apply local commits on top of the updated remote branch to maintain a linear history.

Procedure :

Step 1 : Open the Existing Git Repository

```
cd git_exp1
```

The existing local Git repository is opened to perform remote operations.

Step 2 : Check Current Branch

```
git branch
```

Displays the list of working branches and shows the currently active branch

Step 3 : Fetch Latest Changes from Remote Repository

```
git fetch origin
```

Downloads the latest updates from the remote repository without merging them into the local branch.

Step 4 : Check Repository Status

```
git status
```

Shows whether the local branch is behind the remote branch after fetching updates.

Step 5 : Rebase Local Branch onto Remote Branch

```
git rebase origin/main
```

Reapplies local commits on top of the updated remote branch to keep commit history linear.

Step 6 : Continue Rebase After Conflict Resolution(Optional)

```
git rebase --continue
```

Continues the rebase process after resolving conflicts, if any occur.

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git branch
  feature1
* feature_branch
  main
  master

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git fetch origin

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git status
On branch feature_branch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git_lab/

nothing added to commit but untracked files present (use "git add" to track)

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git rebase origin/master
fatal: invalid upstream 'origin/master'

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git branch -r
  origin/HEAD -> origin/main
  origin/main

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (feature_branch)
$ git rebase origin/main
Current branch feature_branch is up to date.
```

EXPERIMENT – 6 : Collaboration And Remote Repositories

Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.

Git merge is used to combine changes from one branch into another branch. This experiment demonstrates how to merge a feature branch into the master branch using a custom commit message.

Procedure :

Step 1 : Open the Existing Git Repository

```
cd git_exp1
```

The existing local Git repository is opened to perform merge operations.

Step 2 : Switch to Master Branch

```
git checkout master
```

The working branch is switched to **master**, where the feature branch will be merged.

Step 3 : Merge Feature Branch with Custom Commit Message

```
git merge feature-branch -m "Merged feature-branch into
master with custom message"
```

This command merges the **feature-branch** into **master** and records the merge using a custom commit message.

Step 4 : Verify Merge Status

```
git status
```

Displays the repository status and confirms that the merge was completed successfully.

```
MINGW64:/c/Users/DELL/git_lab
DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (feature_branch)
$ git checkout master
Switched to branch 'master'

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git merge feature_branch -m "Merged feature-branch into master with custom message"
Merge made by the 'ort' strategy.
 file1.txt | 1 +
 1 file changed, 1 insertion(+)

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

EXPERIMENT – 7 : Git Tags And Releases

Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.

Git tags are used to mark specific points in a repository's history, usually for releases. A lightweight tag is a simple reference to a commit without additional metadata.

Procedure :

Step 1 : Open the Existing Git Repository

```
cd git_exp1
```

The existing local Git repository is opened to create a tag.

Step 2 : View Commit History

```
git log –oneline
```

Displays the list of commits so that a specific commit can be identified for tagging**.**

Step 3 : Create a Lightweight Git Tag

```
git tag v1.0
```

Creates a lightweight Git tag named v1.0 for the latest commit in the local repository.

Step 4 : Verify the Created Tag

git tag

Displays the list of tags created in the repository.

```
MINGW64:/c/Users/DELL/git_lab
Merge made by the 'ort' strategy.
 file1.txt | 1 +
 1 file changed, 1 insertion(+)

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git status
On branch master
nothing to commit, working tree clean

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git log --oneline
0a77379 (HEAD -> master) Merged feature-branch into master with custom message
8dd7425 (feature_branch) Updated file1.txt in feature1 branch
879337c (feature1) Updated file1.txt in feature1 branch
8bd8e55 Updated file1.txt in feature1 branch
c18fc04 Added file1.txt for first Git experiment
401def7 (main) Initial commit -adding a new file
6a97b06 Initial commit

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git tag v1.0

DELL@DESKTOP-AGGGAP4 MINGW64 ~/git_lab (master)
$ git tag
v1.0
```

EXPERIMENT – 8 : Advanced Git Operations

Write the command to cherry-pick a range of commits from "source-branch" to the current branch.

Git cherry-pick is used to apply specific commits from one branch to another branch. This experiment demonstrates how a range of commits can be selectively applied without merging the entire branch.

Procedure :

Step 1 : Open the Existing Git Repository

```
cd git_exp1
```

The existing local Git repository is opened to perform advanced Git operations.

Step 2 : Switch to the Target Branch

```
git checkout master
```

The branch where the selected commits need to be applied is checked out.

Step 3 : View Commit History of Source Branch

```
git log --oneline source-branch
```

Displays the commit history of the source branch to identify the range of commits.

Step 4 : Cherry-Pick a Range of Commits

git cherry-pick <start_commit>^..<end_commit>

Applies a specific range of commits from source-branch to the current branch.

Step 5 : Verify the Applied Commits

git log –oneline

Verifies that the selected commits have been successfully applied to the current branch.

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master)
$ git log --oneline --all
08b66ec (HEAD -> master, tag: v1.0) Initial commit on master
4fd275b (feature_branch) Added file1.txt for first Git experiment
6644482 Initial commit
b10f114 Delete lab1.txt
e80e47b Initial commit -adding a new file
280441d (main, feature1) Added file1.txt for first Git experiment
07e4fcc Initial commit
b3a0936 (origin/main, origin/HEAD) Initial commit
46f12f7 Delete lab1.txt
0b432ad Initial commit -adding a new file
389576d Initial commit

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master)
$ git cherry-pick 389576d
On branch master
You are currently cherry-picking commit 389576d.
  (all conflicts fixed: run "git cherry-pick --continue")
  (use "git cherry-pick --skip" to skip this patch)
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)

nothing to commit, working tree clean
The previous cherry-pick is now empty, possibly due to conflict resolution.
If you wish to commit it anyway, use:

    git commit --allow-empty

Otherwise, please use 'git cherry-pick --skip'

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ |
```

EXPERIMENT – 9 : Analysing And Changing Git History

Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?

Git allows users to inspect detailed information about any commit using its unique commit ID. This experiment demonstrates how to view commit details such as author, date, and commit message.

Procedure :

Step 1 : Open the Existing Git Repository

```
cd git_exp1
```

The existing local Git repository is opened to analyse commit history.

Step 2 : View Details of a Specific Commit

```
git show <commit_id>
```

Displays detailed information about the specified commit, including author name, date, commit message, and changes made.

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ git log --oneline
08b66ec (HEAD -> master, tag: v1.0) Initial commit on master
280441d (main, feature1) Added file1.txt for first Git experiment
07e4fcc Initial commit
46f12f7 Delete lab1.txt
0b432ad Initial commit -adding a new file
389576d Initial commit

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ git show 0b432ad
commit 0b432ad866fc6ff03f99534e9d64e6e5413ee067
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 20:21:56 2026 +0530

    Initial commit -adding a new file

diff --git a/lab1.txt b/lab1.txt
new file mode 100644
index 0000000..e69de29

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ |
```

EXPERIMENT – 10 : Analysing And Changing Git History

Write the command to list all commits made by the author "JohnDoe" between "2023-01-01" and "2023-12-31."

Git provides powerful filtering options to analyse commit history based on author and date. This experiment demonstrates how to list commits made by a specific author within a defined time period.

Procedure :

Step 1 : List Commits by Author Between Given Dates

git log --author="JohnDoe" -- since="2023-01-01" --until="2023-12-31"

Displays all commits made by the author *JohnDoe* within the specified date range.

```
Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ git log --author="Sowndarya B" --since="2025-01-01" --until="2026-12-31"
commit 08b66ec80a7b0642682fa5a73e6b6bdd49140d4a (HEAD -> master, tag: v1.0)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Wed Jan 7 15:01:35 2026 +0530

    Initial commit on master

commit 280441d04f26cb75087b5de44c2ee94272b0dd06 (main, feature1)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Tue Jan 6 15:13:42 2026 +0530

    Added file1.txt for first Git experiment

Sindhu B@LAPTOP-LAB9ROSE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$
```

EXPERIMENT – 11 : Analysing And Changing Git History

Write the command to display the last five commits in the repository's history**.**

Git maintains a complete history of all commits made in a repository. This experiment demonstrates how to view the most recent commits using Git log options.

Procedure :

Step 1 : Display the Last Five Commits

```
git log -5
```

Displays the most recent five commits along with commit ID, author, date, and commit message.

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ git log -3
commit 08b66ec80a7b0642682fa5a73e6b6bdd49140d4a (HEAD -> master, tag: v1.0)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Wed Jan 7 15:01:35 2026 +0530

    Initial commit on master

commit 280441d04f26cb75087b5de44c2ee94272b0dd06 (main, feature1)
Author: Sowndarya B <soundaryab0402@gmail.com>
Date:   Tue Jan 6 15:13:42 2026 +0530

    Added file1.txt for first Git experiment

commit 07e4fcc30b28baa5d33638b359872fd2655f1305
Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
Date:   Mon Jan 5 21:42:12 2026 +0530

    Initial commit

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$
```

EXPERIMENT – 12 : Analysing And Changing Git History

Write the command to undo the changes introduced by the commit with the ID "abc123".

Git provides commands to safely undo changes introduced by previous commits. This experiment demonstrates how to revert a specific commit without affecting other commits.

Procedure :

Step 1 : Undo Changes Introduced by a Specific Commit

```
git revert abc123
```

Creates a new commit that reverses the changes made by the commit with ID abc123.

Step 2 : Verify Commit History

```
git log –oneline
```

Displays the commit history to confirm that the revert operation was successful

```
Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ git init
Reinitialized existing Git repository in C:/Users/Sindhu B/git_lab/.git/

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|CHERRY-PICKING)
$ touch ok.txt && git add . && git commit -m "Stable"
[master 6a60824] Stable
 Author: soundaryab0402-cloud <soundaryab0402@gmail.com>
 Date: Mon Jan 5 20:19:40 2026 +0530
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 ok.txt

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master)
$ touch error.txt && git add . && git commit -m "Mistaken Commit"
[master 48278b0] Mistaken Commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 error.txt

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master)
$ git log --oneline
48278b0 (HEAD -> master) Mistaken Commit
6a60824 Stable
08b66ec (tag: v1.0) Initial commit on master
280441d (main, feature1) Added file1.txt for first Git experiment
07e4fcc Initial commit
46f12f7 Delete lab1.txt
0b432ad Initial commit -adding a new file
389576d Initial commit

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master)
$ git revert 0b432ad
CONFLICT (modify/delete): lab1.txt deleted in parent of 0b432ad (Initial commit -adding a new file) and modified in HEAD.  Version HEAD of lab1.txt left in tr
ee.
error: could not revert 0b432ad... Initial commit -adding a new file
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config set advice.mergeConflict false"

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|REVERTING)
$ This reverts commit 0b432ada7763436b67231c774coe33d566d9288
bash: This: command not found

Sindhu B@LAPTOP-LAB9R0SE MINGW64 ~/git_lab (master|REVERTING)
$
```

Appendix

Repository Link

https://github.com/soundaryab0402-cloud/4GW24CI049..git